Estructura del backend – NestJS

Oct 28, 2026

Guía de implementación: dónde va cada archivo y qué hace. Las reglas de negocio y el análisis están en el informe; acá solo se indica en qué pieza se implementa cada una.

Capas

Seguimos la estructura del profesor, con dos cambios: **use cases** en lugar de services de aplicación, y **un solo repositorio**, sin persistence adapter. Las dependencias apuntan siempre hacia `domain/`.

Carpeta	Capa	Qué va	Consulta la base
`application/controllers/`	Aplicación	Endpoints REST. Sin lógica	No
`application/use-cases/`	Aplicación	Un use case por comando: busca, llama a la entidad, guarda y publica el evento. Lleva `@Transactional`	A través del repositorio
`application/queries/`	Aplicación	QueryObject (los filtros) y QueryService (consulta de solo lectura que devuelve DTOs)	Sí, con el repositorio de TypeORM directo
`domain/`	Dominio	Entidades con su lógica, value objects, enums, factories, services de dominio, interfaz del repositorio, eventos	**No**
`infraestructure/repositories/`	Infraestructura	Implementa la interfaz con TypeORM	Sí

Carpeta	Capa	Qué va	Consulta la base
`dto/`	—	Datos de entrada y salida de la API	—
`mappers/`	—	Entidad → DTO de respuesta (método `aDto`)	—

En una línea: el controlador recibe, el use case coordina, la entidad decide y el repositorio guarda.

Estructura general

Un módulo de Nest por contexto del análisis, dentro de `src/modules/`, más `common/` para lo transversal. Todos los módulos repiten este molde:

```
src/
├── modules/
│   ├── registro/                ← core: sesiones, series, récord, progreso
│   │   ├── application/
│   │   │   ├── controllers/
│   │   │   ├── use-cases/
│   │   │   └── queries/         ← QueryObjects y QueryServices
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   ├── value-objects/
│   │   │   ├── interfaces/      ← contrato del repositorio
│   │   │   ├── services/        ← services de dominio (no consultan la
base)
│   │   │   ├── factories/
│   │   │   └── events/
│   │   ├── infraestructure/
│   │   │   └── repositories/    ← implementa la interfaz con TypeORM
│   │   ├── dto/
│   │   ├── mappers/
│   │   └── registro.module.ts
│   ├── planificacion/           ← planes, rutinas, dosificaciones,
asignaciones
│   ├── catalogo/                ← ejercicios precargados
│   ├── gestion-usuario/         ← auth/ (login, token), rol/ y usuario/
(cuentas, vínculo)
```

```
|   ├── mediciones/              ← peso corporal y altura
|   └── common/                  ← filtro global de errores
├── app.module.ts
└── main.ts
```

Qué va en `events/`: las clases de los eventos de dominio. Son objetos simples que describen un hecho que ya pasó, con sus datos (por ejemplo, `PlanAsignado`: plan, deportista, vigencia y fecha). No tienen lógica: el use case los crea y los publica después de guardar, con EventEmitter2 (paquete @nestjs/event-emitter).

Una carpeta que un módulo no necesita no se crea: por ejemplo, `catalogo` no tiene `use-cases/` ni `events/`.

Convenciones de nombres

Archivos en `kebab-case` con un sufijo que indica qué son; clases en `PascalCase`, con los nombres del glosario.

Sufijo del archivo	Qué es	Carpeta	Ejemplo
`.controller.ts`	Controlador	`application/controllers/`	`series.controller.ts`
`.use-case.ts`	Use case	`application/use-cases/`	`registrar-serie.use-case.ts`
`.query.ts`	QueryObject	`application/queries/`	`ejercicios.query.ts`
`.query-service.ts`	QueryService	`application/queries/`	`ejercicios.query-service.ts`
`.entity.ts`	Entidad (TypeORM + su lógica)	`domain/entities/`	`sesion-entrenamiento.entity.ts`
`.vo.ts`	Value object	`domain/value-objects/`	`peso.vo.ts`
`.enum.ts`	Enum de dominio	`domain/enums/`	`rol-nombre.enum.ts`
`.repository-interface.ts`	Contrato del repositorio	`domain/interfaces/`	`plan.repository-interface.ts`
`.service.ts`	Service de dominio	`domain/services/`	`detector-record-personal.service.ts`
`.factory.ts`	Factory	`domain/factories/`	`plan-entrenamiento.factory.ts`
`.event.ts`	Evento de dominio	`domain/events/`	`serie-registrada.event.ts`

Sufijo del archivo	Qué es	Carpeta	Ejemplo
`.repository.ts`	Implementación del repositorio	`infraestructure/repositories/`	`plan.repository.ts`
`.dto.ts`	Entrada o salida de la API	`dto/`	`registrar-serie.dto.ts`
`.mapper.ts`	Entidad → DTO de respuesta	`mappers/`	`plan.mapper.ts`

Singular y plural: clases y archivos de entidades en singular (`Serie`); listas y endpoints en plural (`series` , `/api/series`).

Módulo `registro` (core)

```
registro/
├── application/
|   ├── controllers/      series.controller.ts, sesiones.controller.ts,
progreso.controller.ts
|   ├── use-cases/        registrar-serie.use-case.ts, corregir-serie.use-
case.ts
|   └── queries/          sesiones.query.ts, sesiones.query-service.ts,
progreso.query-service.ts
├── domain/
|   ├── entities/         sesion-entrenamiento.entity.ts, serie.entity.ts
|   ├── value-objects/    peso.vo.ts, repeticiones.vo.ts, rir.vo.ts
|   ├── interfaces/       sesion-entrenamiento.repository-interface.ts
|   ├── services/         detector-record-personal.service.ts, calculadora-
progreso.service.ts,
|   |                     calculadora-cumplimiento.service.ts
|   ├── factories/        sesion-entrenamiento.factory.ts
|   └── events/           serie-registrada.event.ts, serie-
corregida.event.ts,
|                         record-personal-superado.event.ts
├── infraestructure/
|   └── repositories/     sesion-entrenamiento.repository.ts
├── dto/                  registrar-serie.dto.ts, corregir-serie.dto.ts
├── mappers/              serie.mapper.ts
└── registro.module.ts
```

Pieza	Tipo	Responsabilidad	Reglas
`SesionEntrenamiento`	Raíz de agregado	Registra y corrige sus series; única puerta de entrada	RN2, RN7, RN8
`Serie`	Entidad interna	Número, ejercicio, peso, repeticiones y RIR	—

Pieza	Tipo	Responsabilidad	Reglas
`Peso`, `Repeticiones`, `RIR`	Value objects	Se validan al crearse	RN5, RN4, RN6
`SesionEntrenamientoFactory`	Factory	Crea la sesión con la primera serie y la fecha del sistema	RN2, RN12
`DetectorRecordPersonal`	Service de dominio	Decide si una serie supera el récord	RN9
`CalculadoraProgreso`	Service de dominio	Peso máximo, 1RM y volumen por sesión	RN16
`CalculadoraCumplimiento`	Service de dominio	Series registradas ÷ objetivo, con tope por ejercicio	—
`ISesionEntrenamientoRepository`	Interfaz	`buscarDelDia`, `buscarPorId`, `buscarSeriesDelEjercicio`, `guardar`	—
`SesionEntrenamientoRepository`	Repositorio	Implementa la interfaz con TypeORM	—
`RegistrarSerieUseCase`	Use case	Verifica el plan vigente → sesión del día (Factory) → registrar → detectar récord → guardar → eventos	RN1, RN3
`CorregirSerieUseCase`	Use case	Busca la sesión → corrige → detecta récord → guarda → eventos	RN1
`SesionesQuery`	QueryObject	Filtros opcionales `desde` y `hasta`	—
`SesionesQueryService`	QueryService	Sesión de hoy con su cumplimiento, historial de sesiones, últimas series de un ejercicio y asistencia para el entrenador	RN17
`ProgresoQueryService`	QueryService	Progreso de un ejercicio, ejercicios con progreso y peso máximo para el entrenador (usa `CalculadoraProgreso`)	RN9, RN16, RN17
`SeriesController`	Controlador	`POST /api/series`, `PATCH /api/series/{id}`	—
`SesionesController`	Controlador	`GET /api/sesiones/hoy`, `GET /api/sesiones`, `GET /api/ejercicios/{id}/ultimas-series`, `GET /api/deportistas/{id}/asistencia`	—
`ProgresoController`	Controlador	`GET /api/progreso`, `GET /api/progreso/ejercicios`, `GET /api/deportistas/{id}/progreso/{ejercicio_id}`	—

Módulo `planificacion`

```
planificacion/
├── application/
|   ├── controllers/      planes.controller.ts, asignaciones.controller.ts
|   ├── use-cases/        crear-plan.use-case.ts, modificar-plan.use-case.ts,
|   |                     duplicar-plan.use-case.ts, asignar-plan.use-case.ts
|   └── queries/          planes.query-service.ts
├── domain/
|   ├── entities/         plan-entrenamiento.entity.ts, rutina.entity.ts,
asignacion.entity.ts
|   ├── value-objects/    dosificacion.vo.ts, rango-repeticiones.vo.ts,
vigencia.vo.ts
|   ├── interfaces/       plan.repository-interface.ts
|   ├── services/         verificador-superposicion.service.ts
|   ├── factories/        plan-entrenamiento.factory.ts
|   └── events/           plan-creado.event.ts, plan-asignado.event.ts
├── infraestructure/
|   └── repositories/     plan.repository.ts
├── dto/                  crear-plan.dto.ts, asignar-plan.dto.ts
├── mappers/              plan.mapper.ts
└── planificacion.module.ts
```

Pieza	Tipo	Responsabilidad	Reglas
`PlanEntrenamiento`	Raíz de agregado	Modifica rutinas y dosificaciones, agrega asignaciones	RN10, RN13
`Rutina`	Entidad interna	Nombre y dosificaciones; busca la dosificación de un ejercicio	RN15 (sin ejercicio repetido)
`Asignacion`	Entidad interna	Deportista y vigencia; sabe si ya empezó	—
`Dosificacion`, `RangoRepeticiones`, `Vigencia`	Value objects	Se validan al crearse; la dosificación se reemplaza entera	RN15, RN10
`PlanEntrenamientoFactory`	Factory	Crea el plan completo y la copia al duplicar	RN10, RN15
`VerificadorSuperposicion`	Service de dominio	Vigencias de un deportista entre todos sus planes	RN11
`IPlanRepository`	Interfaz	`buscarPorId`, `buscarAsignacionesDelDeportista`, `guardar`	—
`PlanRepository`	Repositorio	Implementa la interfaz con TypeORM	—

Pieza	Tipo	Responsabilidad	Reglas
`CrearPlanUseCase`	Use case	Factory → guardar → **Plan creado**	—
`ModificarPlanUseCase`	Use case	Busca → modifica (la entidad verifica RN13) → guarda	RN13
`DuplicarPlanUseCase`	Use case	Busca → Factory crea la copia → guarda	—
`AsignarPlanUseCase`	Use case	Verifica el vínculo con el repositorio de usuarios → superposición → agregar asignación → guardar → **Plan asignado**	RN14, RN11
`PlanesQueryService`	QueryService	Mis planes, detalle de un plan y plan vigente del deportista	RN17
`PlanesController`	Controlador	`POST /api/planes`, `PUT /api/planes/{id}`, `POST /api/planes/{id}/duplicado`, `GET /api/planes`, `GET /api/planes/{id}`, `GET /api/plan-vigente`	—
`AsignacionesController`	Controlador	`POST /api/planes/{id}/asignaciones`	—

Módulo `catalogo`

No tiene comandos: el catálogo viene precargado y el entrenador no agrega ejercicios. Listar y filtrar es una consulta, así que va en un QueryService (lee con el repositorio de TypeORM directo y devuelve DTOs), no en un use case.

```
catalogo/
├── application/
│   ├── controllers/      ejercicios.controller.ts
│   └── queries/          ejercicios.query.ts, ejercicios.query-service.ts
├── domain/
│   ├── entities/         ejercicio.entity.ts (incluye el enum GrupoMuscular)
│   └── interfaces/       ejercicio.repository-interface.ts
├── infraestructure/
│   ├── repositories/     ejercicio.repository.ts
│   └── seed/             ejercicios.seed.ts
├── dto/                  ejercicio-response.dto.ts
├── mappers/              ejercicio.mapper.ts
└── catalogo.module.ts
```

Pieza	Tipo	Responsabilidad
`Ejercicio`	Raíz de agregado	Nombre y grupo muscular (MMII, MMSS, CORE)
`IEjercicioRepository`	Interfaz	`buscarPorId`, `existe` (los usan Planificación y Registro)
`EjercicioRepository`	Repositorio	Implementa la interfaz con TypeORM
`ejercicios.seed.ts`	Seed	Carga los ejercicios precargados
`EjerciciosQuery`	QueryObject	Filtro opcional `grupo_muscular`; lo valida el `ValidationPipe` (422 si no es un grupo válido)
`EjerciciosQueryService`	QueryService	Arma la consulta con QueryBuilder, ordena por grupo muscular y nombre, devuelve DTOs con el mapper
`EjerciciosController`	Controlador	`GET /api/ejercicios?grupo_muscular`, con `@Roles(RolNombre.ENTRENADOR)`

Módulo `gestion-usuario`

Tres submódulos: `auth/` (login y token, sin dominio), `rol/` (los dos roles) y `usuario/` (cuentas y vínculo). Hoy la lógica está en services (`AuthService`, `UsuarioService`, `RolService`); según el criterio del profesor, pasan a use cases.

```
gestion-usuario/
├── auth/                          ← sin domain/ ni infraestructure/
|   ├── application/
|   |   ├── controllers/           auth.controller.ts
|   |   └── services/              auth.service.ts (a migrar a use cases)
|   ├── dto/                       crear-cuenta.dto.ts, ingresar.dto.ts,
refresh-token.dto.ts
|   ├── auth.guard.ts              guard global: valida el token, deja { id,
rol } en req.user y controla @Roles
|   ├── public.decorator.ts        @Public() para register, login y refresh
|   ├── roles.decorator.ts         @Roles(RolNombre.ENTRENADOR),
@Roles(RolNombre.DEPORTISTA)
|   ├── jwt-payload.interface.ts   { sub, rol }
|   └── auth.module.ts
├── rol/
|   ├── application/services/      rol.service.ts
```

```
|   ├── domain/
|   |   ├── entities/             rol.entity.ts
|   |   ├── enums/                rol-nombre.enum.ts (ENTRENADOR,
DEPORTISTA)
|   |   └── interfaces/           rol.repository-interface.ts
|   ├── infraestructure/repositories/   rol.repository.ts
|   └── rol.module.ts
└── usuario/
    ├── application/
    |   ├── controllers/          usuario.controller.ts,
deportistas.controller.ts (a implementar)
    |   ├── services/             usuario.service.ts (a migrar a use cases)
    |   └── use-cases/            vincular-deportista.use-case.ts (a
implementar)
    ├── domain/
    |   ├── entities/             usuario.entity.ts, vinculo.entity.ts (a
implementar)
    |   └── interfaces/           usuario.repository-interface.ts
    ├── infraestructure/repositories/   usuario.repository.ts
    ├── dto/                      usuario-response.dto.ts
    ├── mappers/                  usuario.mapper.ts
    └── usuario.module.ts
```

Pieza	Submódulo	Tipo	Responsabilidad	Reglas
`AuthService`	auth	Service (a migrar a use cases)	Crear cuenta: hashea la contraseña y delega en `UsuarioService`. Ingresar: verifica credenciales y genera los tokens con id y rol	—
`AuthGuard`	auth	Guard global	Valida el token (401), deja `{ id, rol }` en `req.user` sin consultar la base y, si hay `@Roles`, compara el rol (403)	RN17
`@Public()`	auth	Decorador	Rutas sin token: register, login y refresh	—
`@Roles(...)`	auth	Decorador	Indica qué rol puede usar el endpoint	RN17
`AuthController`	auth	Controlador	`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/refresh`	—
`Rol` y `RolNombre`	rol	Entidad y enum	Los dos roles: ENTRENADOR y DEPORTISTA	—

Pieza	Submódulo	Tipo	Responsabilidad	Reglas
`RolService`	rol	Service	Busca el rol por nombre al crear la cuenta	—
`Usuario`	usuario	Raíz de agregado	Datos de la cuenta y rol; crea el vínculo (a implementar)	RN18
`Vinculo`	usuario	Entidad interna (a implementar)	Entrenador y deportista vinculados	—
`IUsuarioRepository`	usuario	Interfaz	`buscarPorId`, `buscarPorEmail`, `guardar`	—
`UsuarioRepository`	usuario	Repositorio	Implementa la interfaz con TypeORM	—
`UsuarioService`	usuario	Service (a migrar a use cases)	Crea el usuario (email repetido → 422) y busca el perfil	—
`UsuarioMapper`	usuario	Mapper	`aDto`: id, nombre, email y rol, nunca la contraseña	—
`VincularDeportistaUseCase`	usuario	Use case (a implementar)	Busca al deportista por email → `vincular()` → guarda	RN18
`UsuarioController`	usuario	Controlador	`GET /api/usuario/perfil`	—
`DeportistasController`	usuario	Controlador (a implementar)	`GET /api/deportistas` (mis deportistas con su plan vigente), `POST /api/deportistas` (vincular por email)	RN18

Roles

El rol viaja en el token: al ingresar, el token lleva el id y el rol del usuario. En cada endpoint se indica con `@Roles(...)` quién lo puede usar.

Qué se controla	Dónde	Ejemplo
Quién puede usar el endpoint	`@Roles(...)` en el controlador; lo controla el `AuthGuard`	`@Roles(RolNombre.ENTRENADOR)` en planes, catálogo y deportistas; `@Roles(RolNombre.DEPORTISTA)` en series, sesiones, progreso y mediciones
A qué datos puede acceder (RN17)	En el use case o en el QueryService	El entrenador solo ve la asistencia, el peso corporal y el peso máximo de **sus** deportistas vinculados; el deportista solo ve sus propias sesiones

El decorador deja pasar a cualquier entrenador; si ese deportista está vinculado con él lo verifica el use case o el QueryService.

Módulo `mediciones`

```
mediciones/
├── application/
│   ├── controllers/      mediciones.controller.ts
│   ├── use-cases/        registrar-medicion.use-case.ts
│   └── queries/          mediciones.query.ts, mediciones.query-service.ts
├── domain/
│   ├── entities/         medicion-corporal.entity.ts
│   ├── value-objects/    peso-corporal.vo.ts, altura.vo.ts
│   └── interfaces/       medicion.repository-interface.ts
├── infraestructure/
│   └── repositories/     medicion.repository.ts
├── dto/                  registrar-medicion.dto.ts
├── mappers/              medicion.mapper.ts
└── mediciones.module.ts
```

Pieza	Tipo	Responsabilidad	Reglas
`MedicionCorporal`	Raíz de agregado	Peso corporal y altura del deportista en una fecha	RN19
`PesoCorporal`, `Altura`	Value objects	Mayores a 0	RN19
`IMedicionRepository`	Interfaz	`buscarDelDia`, `guardar`	—
`MedicionRepository`	Repositorio	Implementa la interfaz con TypeORM	—

Pieza	Tipo	Responsabilidad	Reglas
`RegistrarMedicionUseCase`	Use case	Busca la medición de hoy → la crea o la reemplaza → guarda	RN19 (una por día)
`MedicionesQuery`	QueryObject	Filtros `deportista_id` (para el entrenador), `desde` y `hasta`	—
`MedicionesQueryService`	QueryService	Evolución del peso corporal, propia o de un deportista vinculado	RN17
`MedicionesController`	Controlador	`POST /api/mediciones`, `GET /api/mediciones`	—

Carpeta `common`

Solo lo que usan todos los módulos y no pertenece a ninguno. El guard y los decoradores viven en `gestion-usuario/auth/`, y los eventos en el `domain/events/` de cada módulo.

```
common/
└── filters/        global-exception.filters.ts
```

Pieza	Responsabilidad
`GlobalExceptionFilter`	Convierte todo error a `{ "error": "mensaje" }`. Si es `HttpException`, usa su código (401, 403, 404, 422) y su mensaje; si es otro error, responde 500 genérico y deja el detalle en el log
`ValidationPipe` (en `main.ts`)	Valida DTOs y QueryObjects antes del controlador; si algo falla, responde 422
`DomainException` (a implementar)	Base de las excepciones de reglas de negocio de Plan y Serie, sin nada de HTTP; el filtro las responde con 422

Para después

No van en la estructura base; se suman cuando una US los necesite. El detalle está en el documento de arquitectura.

Pieza	Dónde va	Cuándo se agrega
Helpers	`common/helpers/`	Cuando el mismo código se repita en varios módulos (fechas, paginación, mensajes)
Repositorios en memoria	`test/`	Con los tests de cada use case